

UNIVERSIDAD TECNOLÓGICA DE PEREIRA

Ingeniería de Sistemas y Computación  
Programación III

## Actividad 1: Lógica de Predicados y Prolog

### Integrantes

Julian Guadir

Alejandro ipaz

### Repositorio GitHub:

<https://github.com/julianguadir56-sudo/trabajo1prog3>

### Ejercicio 1: Árbol genealógico

Las relaciones directas (`padre/2` y `madre/2`) se declaran como hechos. Las relaciones de más de una generación (abuelos, tíos, primos, ancestros, etc.) se obtienen únicamente mediante reglas a partir de `progenitor/2`.

Supuestos sobre la imagen: Herbert aparece unido solo a Abraham (línea punteada), por lo que se modela como hijo de Abraham; Ling aparece solo con Selma. La relación `hermanos/2` incluye medios hermanos (comparten al menos un progenitor).

### Código

```
% EJERCICIO 1: ARBOL GENEALOGICO
% Solo las relaciones directas (padre/madre) son hechos.
% Las relaciones de mas de una generacion se obtienen por reglas.
% =====

% ----- Hechos: genero -----
hombre(abraham) .
hombre(herbert) .
hombre(homero) .
hombre(clancy) .
hombre(bart) .
```

```

mujer(mona).
mujer(jacqueline).
mujer(marge).
mujer(patty).
mujer(selma).
mujer(lisa).
mujer(maggie).
mujer(ling).

% ----- Hechos: relaciones directas -----
% padre(Padre, Hijo)
padre(abraham, herbert).
padre(abraham, homero).
padre(clancy, marge).
padre(clancy, patty).
padre(clancy, selma).
padre(homero, bart).
padre(homero, lisa).
padre(homero, maggie).

% madre(Madre, Hijo)
madre(mona, homero).
madre(jacqueline, marge).
madre(jacqueline, patty).
madre(jacqueline, selma).
madre(marge, bart).
madre(marge, lisa).
madre(marge, maggie).
madre(selma, ling).

% ----- Reglas -----
% progenitor(P, H): P es padre o madre de H
progenitor(P, H) :- padre(P, H).
progenitor(P, H) :- madre(P, H).

% hijo/hija
hijo(H, P) :- hombre(H), progenitor(P, H).
hija(H, P) :- mujer(H), progenitor(P, H).

% abuelos (2 generaciones)
abuelo(A, N) :- hombre(A), progenitor(A, P), progenitor(P, N).
abuela(A, N) :- mujer(A), progenitor(A, P), progenitor(P, N).

% hermanos (comparten al menos un progenitor)
hermanos(X, Y) :- progenitor(P, X), progenitor(P, Y), X \== Y.
hermano(X, Y) :- hombre(X), hermanos(X, Y).
hermana(X, Y) :- mujer(X), hermanos(X, Y).

% tios: hermanos de un progenitor
tio(T, S) :- hombre(T), progenitor(P, S), hermanos(T, P).
tia(T, S) :- mujer(T), progenitor(P, S), hermanos(T, P).

% sobrinos
sobrino(S, T) :- hombre(S), progenitor(P, S), hermanos(T, P).
sobrina(S, T) :- mujer(S), progenitor(P, S), hermanos(T, P).

% primos: hijos de hermanos
primos(X, Y) :- progenitor(P, X), progenitor(Q, Y), hermanos(P, Q).
primo(X, Y) :- hombre(X), primos(X, Y).
prima(X, Y) :- mujer(X), primos(X, Y).

% ancestros y descendientes (recursivas, cualquier numero de generaciones)
ancestro(A, D) :- progenitor(A, D).
ancestro(A, D) :- progenitor(A, X), ancestro(X, D).
descendiente(D, A) :- ancestro(A, D).

% ----- Consultas de prueba (resultado esperado)
% ----- % ?- padre(homero, bart). % true
% ?- abuelo(X, bart). % X = abraham ; X = clancy % ?- abuela(X,
lisa). % X = mona ; X = jacqueline % ?- hermano(bart, lisa). %
true
% ?- hermana(X, bart). % X = lisa ; X = maggie % ?-
tio(herbert, bart). % true
% ?- tia(X, bart). % X = patty ; X = selma % ?- tia(marge,

```

```

ling). % true
% ?- primo(bart, ling). % true
% ?- prima(X, ling). % X = lisa ; X = maggie % ?- sobrino(X,
patty). % X = bart
% ?- ancestro(abraham, X). % herbert, homero, bart, lisa, maggie % ?-
descendiente(ling, X). % X = selma ; X = clancy ; X = jacqueline % ?-
abuelo(clancy, ling). % true

```

## Consultas de validación (salida ejecutada en SWI-Prolog)

| Consulta                  | Resultado                                  |
|---------------------------|--------------------------------------------|
| ?- padre(homero, bart).   | true                                       |
| ?- abuelo(X, bart).       | X = abraham ; X = clancy                   |
| ?- abuela(X, lisa).       | X = jacqueline ; X = mona                  |
| ?- hermano(bart, lisa).   | true                                       |
| ?- hermana(X, bart).      | X = lisa ; X = maggie                      |
| ?- tio(herbert, bart).    | true                                       |
| ?- tia(X, bart).          | X = patty ; X = selma                      |
| ?- tia(marge, ling).      | true                                       |
| ?- primo(bart, ling).     | true                                       |
| ?- prima(X, ling).        | X = lisa ; X = maggie                      |
| ?- sobrino(X, patty).     | X = bart                                   |
| ?- ancestro(abraham, X).  | X = bart, herbert, homero,<br>lisa, maggie |
| ?- descendiente(ling, X). | X = clancy ; X = jacqueline ;<br>X = selma |
| ?- abuelo(clancy, ling).  | true                                       |

The screenshot shows the SWI-Prolog GUI window titled "SWI-Prolog -- C:/Users/GUADIR/Downloads/taller1.pl". The window has a menu bar with "File", "Settings", "Tools", "Debug", "GUI", and "Help". The main text area displays the following content:

```

Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.15)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

```

Below the text, two queries are shown with their results:

```

4 ?- abuelo(X, bart).
X = abraham ;
X = clancy ;
false.

5 ?- abuela(X, lisa).
X = mona ;
X = jacqueline ;
false.

```

Los resultados se muestran sin repeticiones y ordenados.

## Ejercicio 2: El Coronel West es un criminal

La ley se formaliza como: para todo X, si X es estadounidense, vende un arma Y a una nación Z y Z es hostil, entonces X es criminal. Los hechos y reglas del enunciado se traducen así:

- Corea del Sur tiene algunos misiles: `misil(m1). posee(corea_del_sur, m1).` • Todos sus misiles fueron vendidos por West: `vende(west, M, corea_del_sur) :- misil(M), posee(corea_del_sur, M).`
- Los misiles son armas: `arma(M) :- misil(M).`
- Un enemigo de EE. UU. es hostil: `hostil(N) :- enemigo(N, estados_unidos).` • West es estadounidense: `americano(west).`

### Código

```
% EJERCICIO 2: EL CORONEL WEST ES UN CRIMINAL
% Ley: es un crimen que un estadounidense venda armas a naciones hostiles.
% =====

% ----- Hechos -----
americano(west).
enemigo(corea_del_sur, estados_unidos).
misil(m1). % "algunos misiles": existe al menos uno
posee(corea_del_sur, m1).

% ----- Reglas -----
% Todos los misiles de Corea del Sur le fueron vendidos por el Coronel West
vende(west, M, corea_del_sur) :- misil(M), posee(corea_del_sur, M).

% Un misil es un arma
arma(M) :- misil(M).

% Un enemigo de Estados Unidos es una nacion hostil
hostil(N) :- enemigo(N, estados_unidos).

% La ley: crimen de un estadounidense que vende armas a naciones hostiles
criminal(X) :- americano(X), arma(Y), vende(X, Y, N), hostil(N).

% ----- Consultas de prueba (resultado esperado) -----
% ?- criminal(west). % true
% ?- criminal(X). % X = west
% ?- vende(west, M, N). % M = m1, N = corea_del_sur
% ?- hostil(corea_del_sur). % true
% ?- arma(m1). % true
```

### Consultas de validación

| Consulta                  | Resultado                 |
|---------------------------|---------------------------|
| ?- criminal(west).        | true                      |
| ?- criminal(X).           | X = west                  |
| ?- vende(west, M, N).     | M = m1, N = corea_del_sur |
| ?- hostil(corea_del_sur). | true                      |
| ?- arma(m1).              | true                      |
| ?- criminal(homero).      | false                     |

**Conclusión:** Prolog demuestra `criminal(west)` encadenando: west es americano, m1 es un misil (por lo tanto un arma) que Corea del Sur posee, West se lo vende a Corea del Sur, y Corea del Sur es

hostil por ser enemigo de EE. UU. Además, `criminal(homero)` da `false`, lo que confirma que la regla no se cumple para cualquiera.

```
6 ?- criminal(west).  
true.
```

```
7 ?- misil(X).  
X = m1.
```

```
8 ?- enemigo(X, estados_unidos)  
|  
| .  
X = corea_del_sur.
```

```
9 ?- vende(west, M, N).  
M = m1,  
N = corea_del_sur.
```

```
10 ?- criminal(homero).  
false.
```

```
11 ?- ▲
```